GRUNDLAGEN

Coders.Eleven – PYTHON ADVANCED

Coders.Eleven PYTHON ADVANCED
Grundlagen

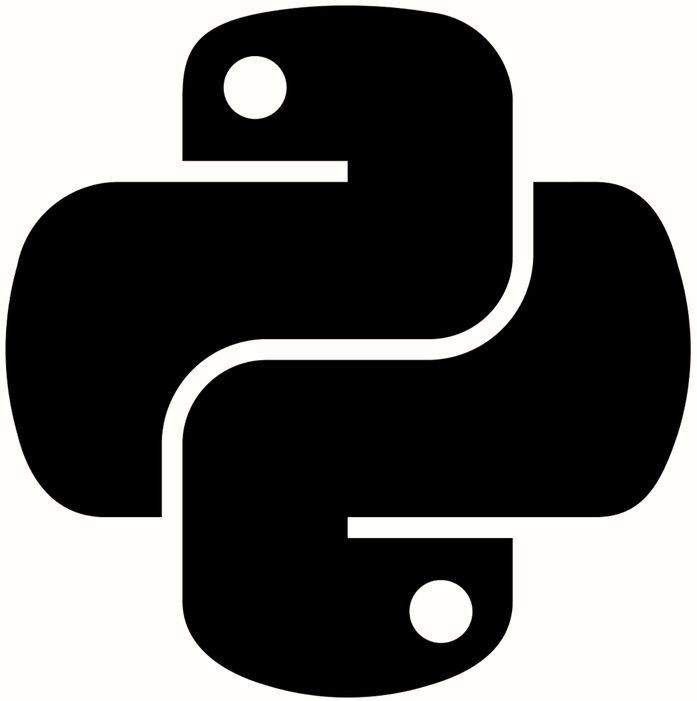
#1 Vergleich

The Zen of Python

Python vs. Java & C#

PEP 8 Style Guide

#2 Datentypen



#3 Variablen & Operatoren

Keywords

Variablen

Operatoren

#4 Kontrollstrukturen

Verzweigungen

Schleifen

Listen

Funktionen

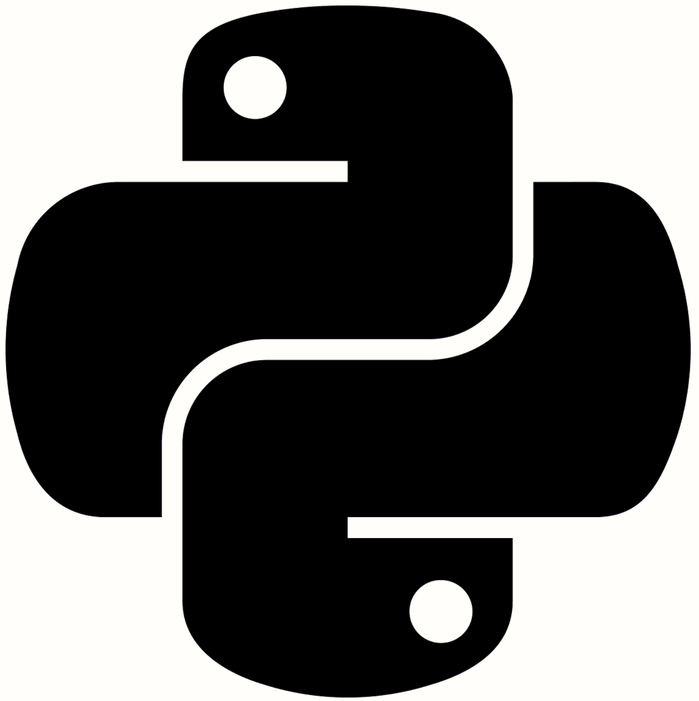
#1 Vergleich

The Zen of Python

Python vs. Java & C#

PEP 8 Style Guide

Terminiertheit



The Zen of Python

- 20 Leitprinzipien für das Design von Python, jedoch wurden nur 19 festgehalten
- Easter Egg: import this

```
Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

- Man schließt daraus, dass Python Code einfach und lesbar sein soll, sodass man sich auf komplexere Probleme fokussieren kann (zB Machine Learning oder Data Science)

Dieselbe Aufgabe – Java

- Zähle, wie viele Wörter mindestens vier Zeichen lang sind

```java
public class Main {
    public static void main(String[] args) {

        String[] woerter = {"Haus", "Tür", "Fenster"};

        int anzahl = 0;

        for(String wort : woerter) {
            if(wort.length() >= 4) {
                anzahl++;
            }
        }
        System.out.println(anzahl);
    }
}
```

Dieselbe Aufgabe – C#

- Zähle, wie viele Wörter mindestens vier Zeichen lang sind

```
public class Zaehler
{
    0 Verweise
    public static void Main()
    {
        string[] woerter = { "Haus", "Tür", "Fenster" };
        int anzahl = 0;

        foreach(string wort in  woerter)
        {
            if(wort.Length >= 4)
            {
                anzahl++;
            }
        }
        Console.WriteLine(anzahl);
    }
}
```

Dieselbe Aufgabe – Python

- Zähle, wie viele Wörter mindestens vier Zeichen lang sind

```python
woerter = ["Haus", "Tür", "Fenster"]
anzahl = 0

for wort in woerter:
    if len(wort) >= 4:
        anzahl = anzahl + 1

print(anzahl)
```

- C# und Java ca. gleich viele Zeilen Code – Python deutlich weniger
- Keine Klasse, main-Methode, Typangaben, geschweifte Klammern, Semikolons

Python vs. Java & C#

- In Python muss es keine Klasse oder main-Methode geben
- Python-Skript kann einen beliebigen Namen haben (Endung ist .py)
- Wenn ein Python-Skript ausgeführt wird, wird alles, was nicht in einer function steht ausgeführt.

- Variablen in Python sind nur Labels, die auf eine Object-Reference im Speicher zeigen.
- Sie sind nicht an einen Datentyp gebunden

Python vs. Java & C#

- Semikolons sind optional in Python
- Zeilenumbruch (\n) markiert das Ende eines Statements

- Geschwungene Klammern werden nicht für Blocks verwendet
- Stattdessen werden Doppelpunkt (:) und Einrückungen kombiniert
  - In Java & C# sind Einrückungen optional
- Zum Einrücken können Tabs oder vier Leerzeichen verwendet werden
  - Jedoch nicht im Code mischen, also entweder oder

- Einzeilige Kommentare werden in Python mit einem Hashtag (#) geschrieben
- Mehrzeilige Kommentare mit “““ oder ‘‘‘ am Anfang und Ende des Kommentars

PEP 8 Style Guide

- Python Enhancement Proposal (PEP) 8 stellt Guidelines zur Verfügung, um guten, konsistenten und lesbaren Python Code zu schreiben - PEP 8
- Achtung: Es handelt sich um Empfehlungen, keine Regeln, siehe A Foolish Consistency is the Hobgoblin of Little Minds

PEP 8 Style Guide - Whitespaces

- PEP 8 empfiehlt, dass man nicht nach Operatoren ausrichten soll:

Yes:

```
x = 1
y = 2
long_variable = 3
```

No:

```
x             = 1
y             = 2
long_variable = 3
```

- PEP 8 empfiehlt, dass man Leerzeichen bei rund um Operatoren machen soll, jedoch nicht wenn in längeren Ausdrücken etwas gruppiert werden soll:

```
# Correct:
i = i + 1
submitted += 1
x = x*2 - 1
hypot2 = x*x + y*y
c = (a+b) * (a-b)
```

```
# Wrong:
i=i+1
submitted +=1
x = x * 2 - 1
hypot2 = x * x + y * y
c = (a + b) * (a - b)
```

PEP 8 Style Guide - Breaking

- PEP 8 empfiehlt eine maximale Zeilenlänge von 79 Zeichen
- Mehrzeilige Ausdrücke nach PEP 8, mithilfe von Klammern („implied line continuation“):

```python
# Correct:
# easy to match operators with operands
income = (gross_wages
          + taxable_interest
          + (dividends - qualified_dividends)
          - ira_deduction
          - student_loan_interest)
```

```python
# Wrong:
# operators sit far away from their operands
income = (gross_wages +
          taxable_interest +
          (dividends - qualified_dividends) -
          ira_deduction -
          student_loan_interest)
```

- Mehrzeilige Strings nach PEP 8:

```
>>> lyrics = "Never gonna give you up,\
...     Never gonna let you down,\
...     Never gonna run around and desert you."
>>> lyrics
'Never gonna give you up,    Never gonna let you down,    Never gonna run around and desert you.'
```

```
>>> lyrics = ("Never gonna give you up, "
...           "Never gonna let you down, "
...           "Never gonna run around and desert you.")
>>> lyrics
'Never gonna give you up, Never gonna let you down, Never gonna run around and desert you.'
```

PEP 8 Style Guide - Breaking

- Für mehrzeilige Bedingungen (zB in while und if) können auch Klammern oder Backslashes verwendet werden:

```
# No extra indentation.
if (this_is_one_thing and
    that_is_another_thing):
    do_something()

# Add a comment, which will provide some distinction in editors
# supporting syntax highlighting.
if (this_is_one_thing and
    that_is_another_thing):
    # Since both conditions are true, we can frobnicate.
    do_something()

# Add some extra indentation on the conditional continuation line.
if (this_is_one_thing
        and that_is_another_thing):
    do_something()
```

PEP 8 Style Guide – True/False

- Nutzen Sie für Wahrheitswerte die verkürzte Form in while und if:

```python
# Correct:
if greeting:
```

```python
# Wrong:
if greeting == True:
```

Worse:

```python
# Wrong:
if greeting is True:
```

- Tipp: Verwenden Sie sinnvolle Namen für boolean Werte, zB is_prime oder has_number

PEP 8 Style Guide – PyCodeStyle

- Mithilfe der Python Command Line Utility pycodestyle kann man überprüfen lassen, ob der Code PEP 8 konform ist
- Beispiel:

```
guess = 5
if guess > 5:
  print("Too large")
  print("Try again")
else:
  print("Ok")
```

- Ausgabe:

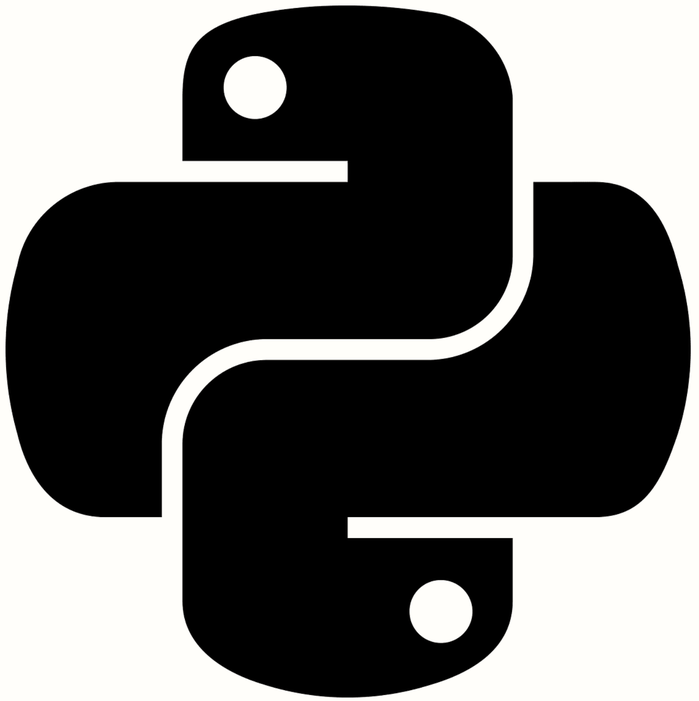
```
test.py:3:3: E111 indentation is not a multiple of four
test.py:4:3: E111 indentation is not a multiple of four
test.py:6:3: E111 indentation is not a multiple of four
```

#2 Datentypen

Arten von Sprache

Sprache umwandeln

Java Ablauf



Datentypen – Überblick

- Python bietet nicht so viele Datentypen wie Java & C# an:
  - Numbers: int, float, complex
  - Booleans: bool
  - Strings: str
- Es gibt noch den Typ „bytes“, wird jedoch kaum verwendet
- Literal: Prägnante und direkte Art einen Wert zu schreiben, zB 558, 2.6, “hello“, True

Datentypen – Numbers & Booleans

- Zahlen können sein:
  - int (ganzzahlig)
  - float (Gleitkommazahlen – äquivalent zu double in Java & C#)
  - complex (Komplexe Zahlen)
- Wahrheitswerte (bool) können True oder False sein
  - Achtung: True und False werden groß geschrieben

Datentypen – Strings

- Zeichenketten (str) sind eine Folge aus Zeichen
- Sie können mit einfachen (‘) oder doppelten (“) Anführungszeichen, aber auch mit dreifachen (‘‘‘) oder dreifachen-doppelten (“““) Anführungszeichen geschrieben werden

- Zwei Strings nebeneinander werden automatisch verbunden:

```
>>> words = "desert" "you"
>>> words
'desertyou'
```

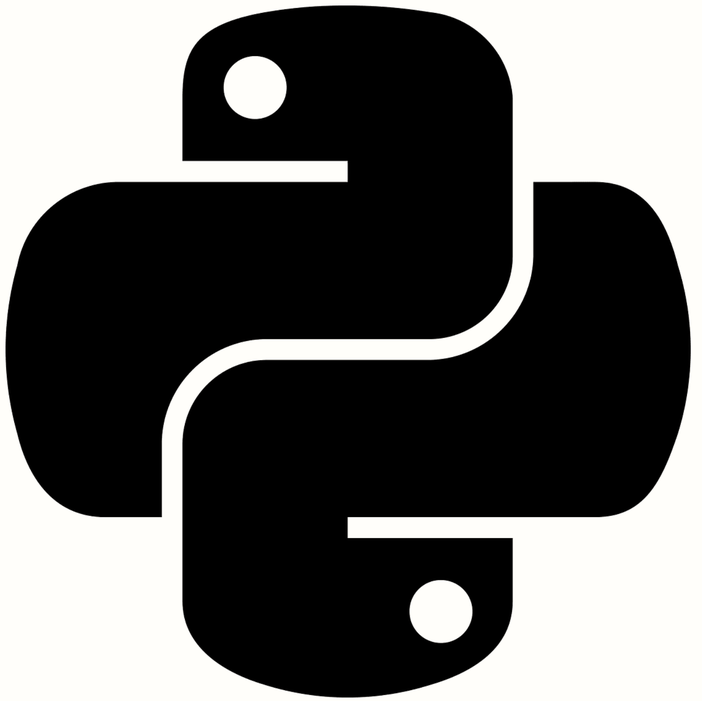
- Achtung: Es gibt keinen Character (char) Datentyp in Python
- Advanced-Hinweis: Sie erinnern sich an die Multiline Comments (Folie 9) – eigentlich sind das keine Kommentare, sondern Strings. Python verarbeitet einen Multiline String als expression (also kein Statement, da nichts zugewiesen oder returnt wird), deshalb passiert nichts damit.

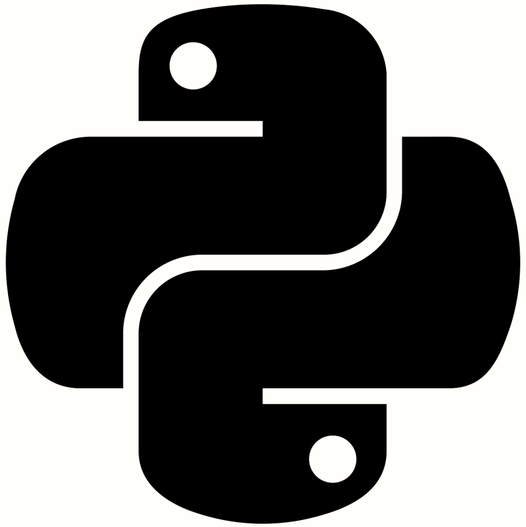
#3 Variablen & Operatoren

Keywords

Variablen

Operatoren





Everything is an object in Python!

Keywords

- Wie Java & C#, hat auch Python eigene Keywords
- Eine Liste aller Keywords bekommt man mit:

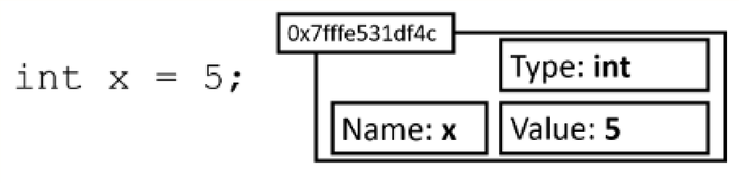
```
>>> import keyword
>>> keyword.kwlist
```

- Hinweis: Vordefinierte Datentypen (zB int, str) sind keine Keywords, sondern Klassen. Man kann diese also theoretisch als Variablennamen verwenden, sollte man aber nicht.

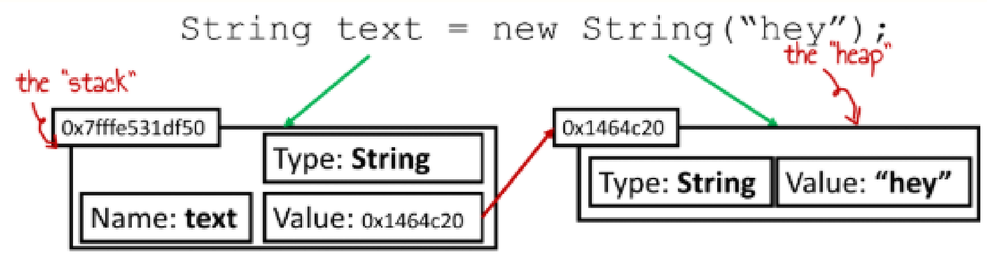
- None
  - Äquivalent zu null in Java & C#
  - None ist ein Objekt (eine Instanz der NoneType-Klasse)

Variablen – Python vs. Java & C#

- In Java & C# stellen Variablen einen reservierten Speicherbereich dar
  - Jede Variable hat einen Datentyp, einen Namen und einen Wert

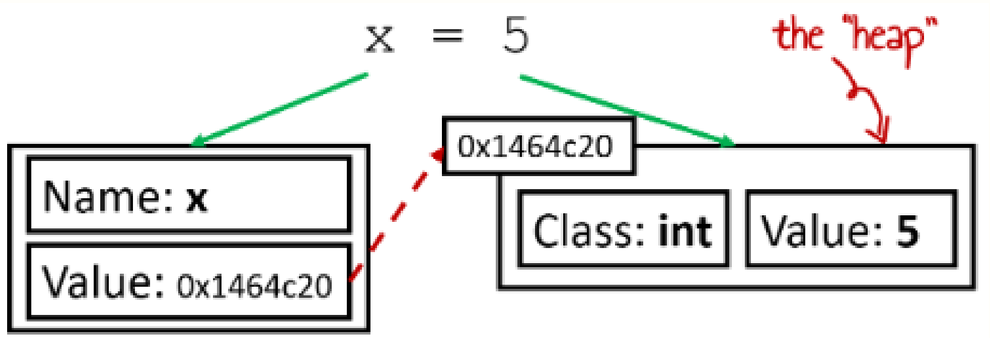


- Stellen Sie sich (Wert)typen in Python NICHT so vor wie Sie es bisher in Java & C# gelernt haben!
- Python-Variablen verhalten sich wie Referenztypen, d.h. sie zeigen auf ein Objekt im Heap
- Variablen in Java & C#: Variable speichert als Wert die Speicheradresse des Objekts, auf welches sie im Heap zeigt. Variable ist gebunden an den Datentyp.



Variablen – Python vs. Java & C#

- In Python ist eine Variable nur ein Label (bzw. Nickname) und zeigt immer auf ein Objekt im Heap
  - Sie erinnern sich: Everything is an object in Python! (Auch Basic Datentypen und None)
  - Anders als in Java & C#, haben Variablen gar keinen Datentyp, sondern nur das Objekt, auf das sie zeigen



- Garbage Collection existiert genauso wie in Java & C#

Objekte in Python

- Nicht vergessen: Everything is an object in Python. Auch Klassen, denn diese erben standardmäßig von der Object-Klasse (wie auch in Java).
- Klasse in Python erstellen:

```
1   class Person:
2       pass
3
4   person = Person()
```

  - Zeile 2: pass ist ein spezielles Statement, welches sagt „mache nichts“. Wird gerne verwendet wenn man einen leeren Block hat
  - Zeile 4: Eine Instanz der Person-Klasse (durch aufrufen des Konstruktors) wird erstellt und der Variable person zugewiesen

Arithmetische Operatoren

- Mathematische Operatoren wie +, -, *, / und % sind gleich wie in Java & C#
  - Achtung: / liefert einen float, ist also keine ganzzahlige Division
    - Dafür gibt es // (entspricht / in Java & C#)
  - Zusätzlich gibt es den Power-Operator **, welcher die Potenz berechnet
    - Also 2**3 = 2^3 = 8
  - Auswertungsreihenfolge:
    1. ( )
    2. **
    3. * // / %
    4. + -
    - Bei gleicher Wertigkeit wird von links nach rechts berechnet
- Achtung: + und * können auch für Strings verwendet werden

Vergleichs- & logische Operatoren

- Die Vergleichsoperatoren <, >, <=, >=, ==, != sind wie in Java & C#
- Jedoch können Ausdrücke verknüpft werden:

```
>>> x = 5
>>> 3 < x < 6
True
>>> 5 < x <= 8
False
```

- Die logischen Operatoren !, &&, || werden in Python ausgeschrieben, d.h. not, and, or.

```
>>> not True
>>> True and False
>>> True or False
>>> x = 7
>>> y = 2
>>> x > 3 and y < 1
>>> not x >= 5 and x < 3      # make sure you understand the
>>> not (x >= 5 and x < 3)    # difference between these two
>>> not 5 + y > 3 * x or 4 == x + 3
```

Typumwandlungen

- Python bietet eingebaute Funktionen, um Werte eines Typs in einen anderen Typ zu konvertieren

```
>>> int('32')
32
>>> int('Hello')
ValueError: invalid literal for int() with base 10: 'Hello'
```

```
>>> str(32)
'32'
>>> str(3.14159)
'3.14159'
```

```
>>> int(3.99999)
3
>>> int(-2.3)
-2
```

```
>>> float(32)
32.0
>>> float('3.14159')
3.14159
```

- Verwenden Sie die Namen solcher „Built-in-Funktionen“ nicht als Variablennamen. Es ist zwar technisch möglich, kann jedoch zu ärgerlichen und schwer zu findenden Fehlern führen.

Zeichenketten

- Operatoren +, *, in und not in sind erlaubt
- Strings sind unveränderbar (immutable – wie auch in Java & C#), d.h. man kann keine einzelnen Character austauschen (zB my_name[7] = “Y“) bzw. append() anwenden.
- Es gibt eine Reihe von String-Methoden, siehe [Docs String-Methoden]

```
>>> my_name = "Josiah"
>>> print(my_name + my_name)
JosiahJosiah
>>> print(my_name * 7)
JosiahJosiahJosiahJosiahJosiahJosiahJosiah
>>> my_name += my_name
>>> print(my_name)
JosiahJosiah
>>>
>>> my_name = "Josiah"
>>> print("x" in my_name)
False
>>> print("x" not in my_name)
True
```

f-strings

- f-strings (formatted string literals) machen die Formatierung von Strings einfacher
  - Ähnlich wie in C#

```
>>> name = "Josiah"
>>> count = 10
>>> possession = "cars"
>>> print(f"{name} has {count} {possession}")
???
```

Josiah has 10 cars

```
>>> pi = 3.14159265359
>>> print(f"PI with three significant digits: {pi:.3}")
PI with three significant digits: 3.14
>>> print(f"PI with three decimal points: {pi:.3f}")
PI with three decimal points: 3.142
>>> print(f"PI with three significant digits, 9-character width is {pi:9.3}")
PI with three significant digits, 9-character width is      3.14
>>> print(f"PI with three significant digits, 9-character width padded with 0 is {pi:09.3}")
PI with three significant digits, 9-character width padded with 0 is 000003.14
```

- Auch die Textausrichtung kann (mit einer fixen Länge) angegeben werden

```
>>> title = "Title"
>>> print(f"| {title : <20} | {title : ^20} | {title : >20} |")
| Title                |        Title         |                Title |
```

Anführungszeichen in einem String

- Dafür gibt es zwei Möglichkeiten:

  1. Wie in Java & C# mit einem Backslash (\):

```
>>> 'the boy\'s mother'
"the boy's mother"
```

  2. Verschiedene Anführungszeichen verwenden (empfohlen):

```
>>> "the boy's mother"
"the boy's mother"
>>> '''the boy's mother'''
'''the boy's mother'''
>>> 'he said, "hello"'
'he said, "hello"'
```

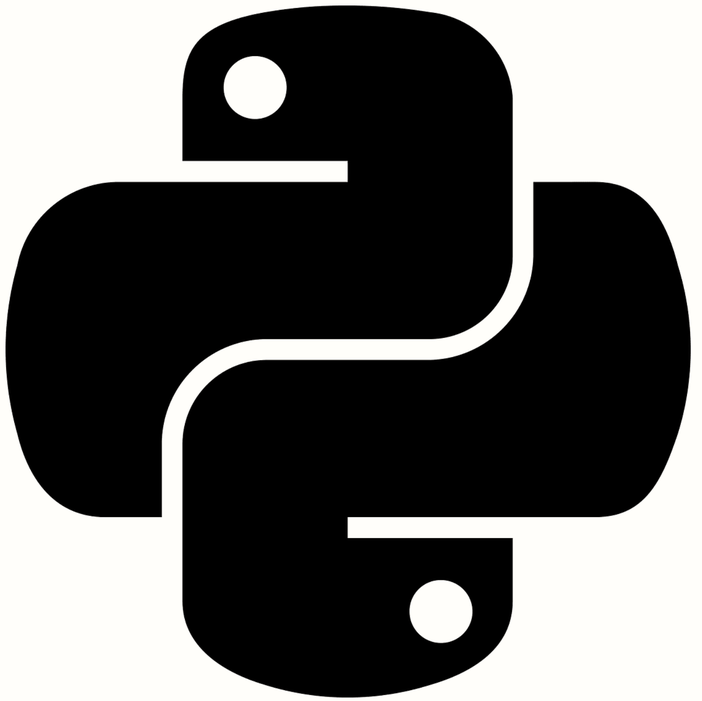
#4 Kontrollstrukturen

Verzweigungen

Schleifen

Listen

Funktionen



Verzweigungen

- Neben den bekannten if und if-else Statements gibt es in Python auf das if-elif-else Statement (wie if-else if-else in Java & C#):

```
if user_guess == 42:
    print("Correct!")
elif user_guess < 42:
    print("Too low")
else:
    print("Too high")
```

- Seit Python 3.10 gibt es das match … case Statement (ähnlich wie ein switch in Java & C#):

```
key = "c"

match key:
    case "a":
        result = 1
    case "b":
        result = 2
    case "c":
        result = 3
    case _:
        result = -1

print(result)
```

Schleifen

- Die while-Schleife in Python ist äquivalent zu jener aus Java & C#
- Die do-while-Schleife existiert in Python nicht
- Die for-Schleife ist in Python als for-each-Schleife implementiert
- Vergleich Python vs. Java:

```python
words = ["I", "have", "a", "dream"]
for word in words:
    print(word)
```

```java
String[] words = new String[] {"I", "have", "a", "dream"};

for (String word : words) {
    System.out.println(word);
}
```

Range

- Möchte man eine „klassische“ for-Schleife haben, kann man über eine range iterieren
- Ein range-Objekt ist ein iterable, das automatisch nach jeder Iteration den counter aktualisiert

```
for i in range(0, 10):
    print(i)
```

```
for i in range(10):
    print(i)
```

- Range(0, 10) generiert eine Folge von Zahlen von 0 bis 9
- Der erste Parameter ist optional und per default 0
- Es gibt einen optionalen dritten Parameter, welcher das increment bestimmt

```
for i in range(0, 10, 2):
    print(i)
```

- Advanced-Tipp: numbers = list(range(0, 10)) liefert eine Liste mit Zahlen.

Listen

- In Python gibt es keine Arrays mit unveränderbarer Größe, dafür aber Listen (wie Listen in C# bzw. ArrayList in Java)
- Eine Liste kann verändert werden und ist in ihrer Größe flexibel

```
>>> students = ["Abraham", "Bella", "Connor", "Da-ming", "Enya"]
```

- Theoretisch kann man in einer Liste mehrere verschiedene Typen speichern (eher vermeiden)

```
>>> mixed_buffet = [1, "Two", 3.03, 4j]
```

- Man kann auch Listen in Listen in Listen usw. erzeugen

```
>>> list_in_list = [1, 2, [3, 4, [5, 6, 7], 8], 9, [10, 11]]
```

- Mit dem list()-Konstruktor kann man ein neues list-Objekt erstellen

```
>>> x = list("abc")    # converts the string "abc" into a list ["a", "b", "c"]
```

- Mit len(list_name) erhält man die Größe der Liste

Listen – Auf Elemente zugreifen

- Sie können auf ein Element der Liste so zugreifen, wie Sie es in Java & C# bei Arrays machen:

```
>>> students = ["Abraham", "Bella", "Connor", "Da-ming", "Enya"]
>>> print(students[0]) # First element
Abraham
>>> print(students[1]) # Second element
Bella
```

- Python erlaubt auch negative Indizes, wodurch Listen „von hinten“ ausgelesen werden können:

```
>>> print(students[-1]) # First element from the end
Enya
>>> print(students[-2]) # Second element from the end
Da-ming
```

0	1	2	3	4
Abraham	Bella	Connor	Da-ming	Enya
-5	-4	-3	-2	-1

- Zugriff bei verschachtelten Listen:

```
>>> x = [1, 2, [3, 4, 5], 6]
>>> print(x[2])
[3, 4, 5]
>>> print(x[2][1])
4
```

Listen – Methoden

- Elemente am Ende hinzufügen mithilfe von append():

```
>>> numbers = [] # this creates a new empty list
>>> numbers.append(6)
>>> print(numbers)
[6]
>>> numbers.append(2)
>>> print(numbers)
[6, 2]
```

- Mehrere Elemente hinzufügen mithilfe von extend():

```
>>> numbers = [1, 2, 3]
>>> numbers.extend([4, 5, 6])
>>> print(numbers)
[1, 2, 3, 4, 5, 6]
```

- Elemente löschen mit dem del-Operator:

```
>>> numbers = [1, 2, 3]
>>> print(numbers)
[1, 2, 3]
>>> del numbers[1]
>>> print(numbers)
[1, 3]
```

- Methoden für list-Objekte: Docs List-Methoden

Listen – Operatoren

- Die Operatoren + und * wurden für Listen überladen
- Man kann also zwei Listen „addieren“:

```
>>> print([1, 2] + [3, 4, 5])
???
```

[1, 2, 3, 4, 5]

- Aber auch „multiplizieren“:

```
>>> print([1, 2] * 3)
???
```

[1, 2, 1, 2, 1, 2]

- Der in-Operator liefert einen bool und überprüft, ob ein Objekt in der Liste enthalten ist:

```
>>> print(3 in [1, 2, 3, 4])
True
>>> print(3 not in [1, 2, 3, 4])
False
>>> print("snake" in ["I", "am", "not", "afraid", "of", "Python"])
False
```

Listen – sort vs. sorted

- list.sort sortiert in-place und returnt None
  - Oftmals in Python, dass Funktionen, die etwas in-place machen None returnen, damit sichtbar ist, dass kein neues Objekt erstellt wurde
- sorted(list) erzeugt eine neue Liste und returnt diese. Funktioniert auch für andere iterable-Objekte, gibt aber immer eine Liste zurück
- Beide Methoden haben zwei optionale Parameter:
  - reverse: Wenn True, wird absteigend sortiert
  - key: Wonach soll sortiert werden? (zB len)

Listen – sort vs. Sorted Beispiel

```
>>> fruits = ['grape', 'raspberry', 'apple', 'banana']
>>> sorted(fruits)
['apple', 'banana', 'grape', 'raspberry']  ❶
>>> fruits
['grape', 'raspberry', 'apple', 'banana']  ❷
>>> sorted(fruits, reverse=True)
['raspberry', 'grape', 'banana', 'apple']  ❸
>>> sorted(fruits, key=len)
['grape', 'apple', 'banana', 'raspberry']  ❹
>>> sorted(fruits, key=len, reverse=True)
['raspberry', 'banana', 'grape', 'apple']  ❺
>>> fruits
['grape', 'raspberry', 'apple', 'banana']  ❻
>>> fruits.sort()                          ❼
>>> fruits
['apple', 'banana', 'grape', 'raspberry']  ❽
```

1. Neue Liste an Strings – alphabetisch sortiert
2. Originale Liste unverändert
3. Alphabetisch sortiert, jedoch rückwärts
4. Neue Liste, sortiert nach der Länge (stabil!)
5. Sortiert nach Länge, jedoch absteigend (trotzdem stabil!)
6. Original unverändrt
7. Liste wird in-place sortiert und None returnt
8. Originale Liste sortiert

Funktionen

- Funktionen sind in Python nicht an eine Klasse gebunden, wie zB in Java
- Es muss kein Rückgabetyp und kein Datentyp für die Parameter angegeben werden
  - Gibt es keinen return-Wert, wird None zurückgegeben
  - Das keyword def kennzeichnet den Beginn einer Funktion
- Es kann zwar nur ein Objekt zurückgegeben werden, jedoch …
  - … Everything is an object in Python … zB auch Listen, Tupel, etc.

```
def compute_something(x, y, z):
    a = x + 2*y
    a = z*a + 3*x
    return a

result = compute_something(5, 6, 3)
print(result)
```

Positional and default arguments

- Um eine Funktion aufzurufen, werden positional arguments verwendet, d.h. Werte werden den Parametern anhand ihrer Position übergeben (wie auch in Java & C#)
- Python erlaubt es, default arguments zu übergeben, sodass nicht jeder Parameter gesetzt werden muss

```
def do_something(a, b, c=2, d="great", e=5, f="john", g="python", h=-1, i=0, j=[]):
    print(f"{a}; {b}; {c}; {d}; {e}; {f}; {g}; {h}; {i}; {j}")
    return None
```

- Die ersten beiden Parameter müssen Werte bekommen, alle anderen sind optional

```
>>> x = do_something("first", "second")
first; second; 2; great; 5; john; python; -1; 0; []

>>> x = do_something("first", "second", "arg for c", "arg for d")
first; second; arg for c; arg for d; 5; john; python; -1; 0; []
```

- Achtung: default parameters müssen immer nach allen „normalen“ Parameter stehen

Keyword arguments

```
def do_something(a, b, c=2, d="great", e=5, f="john", g="python", h=-1, i=0, j=[]):
    print(f"{a}; {b}; {c}; {d}; {e}; {f}; {g}; {h}; {i}; {j}")
    return None
```

- Optionale Parameter können übersprungen werden, indem keyword arguments verwendet werden

```
>>> x = do_something("first", "second", i=2, f="josiah")
first; second; 2; great; 5; josiah; python; -1; 2; []
```

- Die verpflichtenden Parameter müssen jedoch trotzdem zuerst übergeben werden

```
>>> x = do_something(d="no good", "first", "second")
  File "<stdin>", line 1
SyntaxError: positional argument follows keyword argument
```

Built-in functions

- Python bietet viele built-in-Funktionen an, siehe [Docs Built-in functions]
  - Einige davon sind eigentlich Klassen und werden in der Dokumentation auch als solche betitelt

```
>>> print(sum([2, 3]))
>>> print(min([4, 1, 7]))
>>> print(max([4, 1, 7]))
>>> print(abs(-4))           # absolute value
>>> print(pow(2, 4))         # 2 to the power of 4
>>> print(round(5.6))        # round to nearest integer
>>> print(round(5.678, 2))   # round to nearest 2d.p.
```

```
>>> all_true = all([True, True, True, True])
>>> print(all_true)
True
>>> all_true = all([True, False, True, True])
>>> print(all_true)
False
>>> any_true = any([True, False, True, True])
>>> print(any_true)
True
```

```
>>> numbers = [1, 5, 3, 7, 8, 2]
>>> sorted_numbers = sorted(numbers)
>>> print(sorted_numbers)
[1, 2, 3, 5, 7, 8]
>>> reversed_numbers = reversed(numbers)  # reversed returns an `iterator`.
>>> print(list(reversed_numbers))  # you will need to convert it to a list if you need a list
[2, 8, 7, 3, 5, 1]
```

Scope

- Welchen Output liefert folgender Code?

```
>>> def absolute_value(number):
...     y = y + 2
...     print(y)
...     if number < 0:
...         return -number
...     return number
...
... y = 10
... result = absolute_value(5)
```

```
Traceback (most recent call last):
  File "<input>", line 9, in <module>
  File "<input>", line 2, in absolute_value
UnboundLocalError: cannot access local variable 'y' where it is not associated with a value
```

Literaturverzeichnis

- Python for Java programmers - Josiah Wang, Imperial College London
- Python Core Lessons - Josiah Wang, Imperial College London
- Fluent Python, 2nd Edition, Luciano Ramalho, 2021
- Python for Everybody: Exploring Data in Python 3, Charles Severance, 2016